

RAG 완전정복 — 1~4주차 학습 리포트

(처음 보는 사람도 이것만 읽으면 5주차로 바로 넘어갈 수 있게 정리함)

0. 이 리포트가 뭐가

이 글은 RAG라는 기술을 바닥부터 쌓은 학습 기록이다.

- RAG = Retrieval-Augmented Generation = "검색해서(Retrieval) 찾은 자료로 보강해(Augmented) 답을 생성(Generation)하는" 기술.
- 사전지식 0을 가정한다. 비유 → 원리 → 숫자 예시 순으로 간다.
- 마지막(8장)에 5주차(평가)로 가는 다리를 놓는다.

1. 큰 그림: RAG가 왜 필요한가

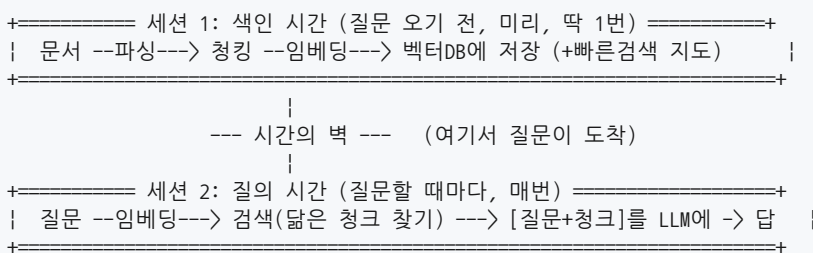
문제: 챗봇(LLM)은 똑똑하지만 모든 걸 정확히 외우지 못한다. 모르는 걸 물으면 그럴듯하게 지어낸다(환각, hallucination). 특히 회사 내부 문서, 최신 법령, 특정 분야 자료는 모른다.

해결: 답하기 전에 진짜 자료를 찾아서 보여주고, 그걸 보고 답하게 한다.

비유: 시험 볼 때 머릿속 기억에만 의존하지 말고, 허용된 컨닝페이퍼(원본 자료)를 펴서 베껴 답하라는 규칙. 그 컨닝페이퍼를 자동으로 찾아 붙여주는 게 RAG다.

2. RAG의 뼈대: "2개의 세션(시간대)"

RAG에서 가장 중요한 개념. 모든 작업은 두 개의 다른 시간대에 나뉘어 일어난다. 이걸 안 나누면 영원히 헛갈린다.



- 세션 1(색인): 비싸고 느리지만 딱 한 번만 한다. 도서관을 미리 짓는 단계.
- 세션 2(질의): 싸고 빠르다. 질문마다 도서관에서 찾아 답한다.

핵심 원칙: 청킹·임베딩 같은 준비는 세션 1에서 끝난다. 질문이 와도 문서를 다시 자르지 않는다. 질문 때 바꿀 수 있는 건 '검색 방식'이지 '청킹'이 아니다.

3. 단계별 상세

3-1. 파싱 (Parsing)

원본 문서(PDF, 워드, 표 등)에서 **순수한 글자만** 뽑아내는 일. 컴퓨터가 읽을 수 있는 텍스트로 변환.

3-2. 청킹 (Chunking)

긴 글을 **의미 단위 토막(chunk)**으로 자르는 일. 왜 자르나? 문서 전체를 통째로 LLM에 넣을 수 없고(너무 큼), 질문과 관련된 부분만 골라 쓰려면 잘게 나눠야 한다.

트레이드오프(양날의 검):

청크가 너무 작으면 -> 정밀하지만 '맥락'이 끊긴다 (앞뒤 문장이 잘림)
청크가 너무 크면 -> 맥락은 살지만 관련 없는 '노이즈'가 섞인다

중요한 교정: "작은 청크 = 비용 증가"는 **틀린 생각**이다. 비용은 LLM에 넣는 토큰에서 발생하고, 작은 청크는 오히려 호출당 싸다. 작은 청크의 진짜 문제는 **맥락 단절**이다.

겹침(overlap): 토막을 자를 때 경계에서 문장이 끊기지 않도록 **앞 토막의 끝부분을 다음 토막에 조금 겹쳐서** 넣는다. 맥락 보존이 목적.

통찰: **최적 청크 크기는 고정값이 아니다.** 문서 성격·질문 유형에 따라 달라지는 '조건 의존적' 값이다.
(예: 법조문은 '조문' 자체가 자연스러운 의미 경계라 그걸로 자르면 맥락이 안 끊긴다.)

3-3. 임베딩 — "의미를 숫자로 만들기" (제조)

임베딩(embedding) = 텍스트를 '의미를 담은 숫자 목록(벡터)'으로 바꾸는 일.

- 원리(분포 가설): "비슷한 맥락에 등장하는 단어는 비슷한 의미를 가진다." '강아지'와 '개'가 비슷한 벡터를 갖는 이유는, 둘 다 "귀엽다/산책/짖는다" 같은 **이웃 단어가 거의 같기 때문이다**. 기계는 의미를 '이해'하는 게 아니라 **이웃을 세서 흉내 낸다**.

숫자로 보기 (축 2개짜리 장난감 예시): 축1=동물스러움, 축2=탈것스러움 이라 하면,

강아지 = [9, 1] (동물 9, 탈것 1)
고양이 = [8, 2]
자동차 = [1, 9]

의미가 비슷하면 숫자(벡터)도 비슷해진다. (진짜 임베딩은 축이 1536개쯤이라 사람이 못 읽을 뿐, 원리는 같다.)

3-4. 코사인 — "두 벡터가 가까운지 재기" (측정)

임베딩이 '만들기'라면, 코사인 유사도는 '재기'다. 둘은 다른 단계다.

- 코사인은 두 벡터의 **방향(각도)**만 본다. 길이(크기)는 무시한다.
- 같은 방향(0°)=1, 직각(90°)=0, 반대(180°)=-1.

숫자로 계산 (공식: 두 벡터 곱의 합 ÷ 각 길이의 곱):

강아지[9,1] vs 고양이[8,2]:
곱의합 = $9 \times 8 + 1 \times 2 = 74$
코사인 = $74 \div (9.06 \times 8.25) \approx 0.99 \rightarrow$ 매우 비슷 [0]

강아지[9,1] vs 자동차[1,9]:
곱의합 = $9 \times 1 + 1 \times 9 = 18$
코사인 = $18 \div (9.06 \times 9.06) \approx 0.22 \rightarrow$ 거의 무관 [X]

왜 길이를 버리고 방향만 보나? '개'를 1번 쓴 글 [9,1]과 100번 쓴 긴 글 [90,10]은 **주제가 같다**. 길이만 다르다.

코사인(방향만): $\approx 0.999 \rightarrow$ "같은 주제" 정답 [0]
직선거리(길이까지): $\approx 81.5 \rightarrow$ "완전 다른 글" 오답 [X]

코사인은 빈도·길이를 '고려'하는 게 아니라 '무시'한다. 그게 핵심이다. "얼마나 길게/자주 썼나"를 버리고 "무엇에 관한 글인가"만 남긴다.

3-5. 철칙: "같은 자(尺)로 재라"

임베딩 모델은 여러 개(OpenAI, Cohere, BGE 등)이고, 모델마다 좌표계가 다르다. 모델 A의 [0.2,0.9]와 모델 B의 [0.2,0.9]는 전혀 다른 의미.

색인할 때 쓴 임베딩 모델 = 질의할 때 쓰는 임베딩 모델. 반드시 동일. 다르면 비교 자체가 무의미해진다. (이걸 어기는 게 실무 RAG 1위 버그.)

3-6. 벡터DB와 ANN — "수백만 개 중에서 빨리 찾기"

질문 벡터 1개 vs 저장된 청크 벡터 100만 개. 가장 가까운 걸 찾으려면?

- **정직한 방법(전수조사):** 100만 개 전부와 코사인 계산. 정확하지만 너무 느림.
- **ANN (Approximate Nearest Neighbor, 근사 최근접):** "완벽한 1등"을 포기하고 "충분히 가까운 것들"로 만족.

정확도 1~2% 내준다 → 속도 수백~수천 배 얻는다

- 어떻게? 질문 오기 전에 벡터들을 '지도(구조)'로 정리해두고, 질문 오면 유망한 구역만 보고 나머진 건너뛰다. (도서관 분류표로 해당 서가만 가는 것.)
- **정체:** 그래서 벡터DB(Pinecone, Milvus, Faiss, Chroma 등) = 저장창고가 아니라 '근사 검색 엔진'이다.

3-7. 질의 시간 — 실제로 답이 나오는 과정

- 1) 사용자가 질문을 친다 (예: "소화기 설치 기준은?")
- 2) 질문도 '같은 모델'로 임베딩 → 벡터로 변환
- 3) 벡터DB에서 ANN으로 '답은 청크 top-k개'를 찾음
- 4) [지시문 + 찾은 청크 텍스트 + 질문]을 하나로 묶어 LLM에 전달
- 5) LLM이 그 청크를 보고 답함

가장 중요한 깨달음 2가지:

- (a) 청크는 '몰래' 붙는다. 사용자는 질문 한 줄만 보내지만, RAG 시스템이 중간에서 검색한 청크를 질문 뒤에 붙여서 LLM에 보낸다. 사용자 눈엔 안 보인다.
- (b) LLM에는 '벡터'가 아니라 '텍스트(글자)'가 들어간다. 벡터(임베딩)는 '어느 청크를 고를까'를 정하는 검색 단계에서만 쓰이고, 청크가 정해지면 임무 끝. LLM은 사람이 읽는 글자를 받아 그걸 컨닝페이퍼처럼 보고 답한다.
- 이래서 LLM을 다시 훈련시킬 필요가 없다. 새 자료가 생기면 벡터DB만 갱신하면, 다음 질문 때 새 청크가 붙어 들어간다.

4. 자주 하는 착각 (반드시 교정할 것)

착각	진실
"질문에 따라 청킹을 다르게 하면 되지"	청킹은 세션 1(미리) 고정. 질문 때 바꾸는 건 검색 방식이지 청킹이 아니다. 굳이 다른 청킹이 필요하면 여러 인덱스를 미리 만들어두고 골라(라우팅) 쓴다.

착각	진실
"코사인은 임베딩 방식이다"	아니다. 임베딩=벡터 만들기, 코사인=두 벡터 재기. 다른 단계.
"질문·체크·LLM입력 다 같은 임베딩?"	임베딩이 일어나는 곳은 2곳뿐(색인·질의)이고 둘은 동일 모델. LLM에 넣는 건 임베딩이 아니라 텍스트.
"모든 걸 벡터DB에 넣는다"	GraphRAG는 저장소가 2개(벡터DB + 그래프). 그래프의 선은 벡터가 아니라 개체 간 관계다.
"ANN은 정확검색이다"	ANN은 일부러 근사. 정확도 조금 내주고 속도를 폭증시킨다.

5. 고도화 지도 (기본 RAG를 넘어서)

철칙: 고도화 = 기법 쌓기가 아니라, 어디가 약한지 '측정'하고 그 지점만 고치는 것. 측정(5주차 평가) 없는 고도화는 미신이다.

ROI 높은 순서:

0. 평가/하네스 구축 <- 측정 없으면 나머지 다 무의미 (5주차)
1. 재랭킹 + 혼합검색 <- 가장 싸고 효과 큼
2. 질문 변형 (rewrite/HyDE) <- 입구 품질 개선
3. 자기교정 페루프 (CRAG) <- 신뢰성 (6주차)
4. 구조 전환 (GraphRAG/Agentic) <- 비싸다. 앞 단계로 안 풀릴 때만 (4·8주차)

- **혼합검색(hybrid):** 의미검색(벡터) + 키워드검색(BM25)을 동시에. 임베딩이 약한 '정확한 단어·번호'를 키워드가 보완.
- **재랭킹(re-ranking):** 1차로 ANN이 후보 50개를 빠르게 긁고, 2차로 더 똑똑한 모델이 정밀 채점해 상위 5개만 남김. 인덱스 안 늘리고 품질을 크게 올리는 최고 가성비 고도화.
- **HyDE:** 질문으로 '가상의 모범답안'을 먼저 써보고, 그 답안을 임베딩해 검색(질문보다 답이 정답 체크와 더 닮았다는 역발상).
- **쿼리 라우팅:** 질문 종류를 보고 알맞은 인덱스/도구로 분기. 단, 자료의 '종류'가 진짜 다를 때만 유효하고, 같은 자료를 체크 크기만 다르게 한 라우팅은 보통 손해.

6. GraphRAG (4주차) — 임베딩이 구조적으로 못 푸는 질문

6-1. 왜 필요한가: 닮음 ≠ 관계

임베딩은 "닮음(유사도)"만 잴 수 있다. 그런데 세상엔 안 닮았는데 강하게 연결된 것이 많다.

"민준" vs "서연": 유사도 낮음(다른 사람), 하지만 관계 강함(사귀) → 임베딩은 '무관'으로 오판

여러 관계를 이어야 답이 나오는 질문(예: "민준과 같은 동아리이면서 민준이 호감 있는 사람은?")은 답이 한 체크에 없고 관계의 사슬에 있어서, 임베딩 검색이 구조적으로 못 푼다.

6-2. 지식을 담는 그릇 3종

표(table): 정해진 칸 → '조건'에 강함 (뻔뻔)
 벡터: 흩어진 점들 → '닮음'에 강함 (연결 모름)
 그래프: 점+선 → '연결'에 강함 (선 따라 걸기 가능)

GraphRAG는 '연결' 질문 때문에 그래프를 쓴다.

6-3. 그래프 만들기 (색인 시간, LLM이 함)

1 **삼중쌍 추출(쉬움)**: LLM이 청크를 읽고 (개체)—[관계]→(개체) 형태로 뽑음.

예: "민준은 사진동아리 소속" → (민준)—[소속]→(사진동아리)

- 1 **개체 통합 / entity resolution (*어려움)**: "민준"="민준이"="그 남자애"처럼 흩어진 **같은 개체를 하나의 점으로 '포개는' 일**. (선을 긋는 게 아니라 중복된 점을 합치는 것.) 이게 실패하면 점이 쪼개져 연결이 끊기고 답이 산으로 간다.
- 2 **온톨로지(ontology)**: 개체·관계의 '타입 설계도'(예: 개체=[인물·장소·동아리], 관계=[소속·사귀·선후배]). 추출이 제멋대로 안 되게 잡아주는 틀.
- 3 **창발(emergence)**: 통합이 끝나면 어느 청크에도 없던 연결이 그래프에 나타난다. 이게 임베딩이 절대 못하는 일.

6-4. 저장소는 2개다

[참고 A: 벡터DB] 청크→벡터. 용도=시작점 찾기(답음)
[참고 B: 그래프] 개체+관계. 용도=연결 따라 걷기
+- 같은 개체 id로 서로 연결됨 (한 덩어리가 아니라 별개 참고 2개) --+

6-5. 질의 시간 (걷기)

- 1 시작점 찾기: 질문 속 '민준' 노드를 찾음 (보통 벡터검색으로!)
- 2 걷기(traverse): 선을 따라 이웃으로 확장
- 3 모으기: 걸어서 만난 점·선을 '서브그래프'로 묶음
- 4 텍스트로 풀어 LLM에 먹임 → 답

공통점: 일반 RAG든 GraphRAG든 결국 "관련 정보를 모아 텍스트로 LLM에 먹인다."

차이점: 일반 RAG는 **답은 청크를**, GraphRAG는 **연결된 서브그래프를** 모은다.

실무에선 **벡터 + 그래프를 함께 쓴다**(시작점은 벡터로, 확장은 그래프로).

6-6. 전역(global) 질문 — 커뮤니티 요약

- **local 질문**("민준의 호감 상대는?"): 시작점에서 몇 걸음 걸으면 끝.
- **global 질문**("이 이야기의 핵심 갈등은?"): **출발할 노드가 없고, 전체가 답이라 그래프가 너무 커서 통째로 LLM에 못 넣음.**
- **해법(Microsoft GraphRAG):**
 - 1 **커뮤니티 탐지**: 그래프를 '뿔뿔하게 뭉친 무리'로 자동 분할(Leiden 등).
 - 2 **무리별 요약**: LLM이 각 무리를 미리 요약. 작은 요약 → 더 큰 요약으로 **계층화**.
 - 3 전역 질문이 오면 무리 요약들을 모아 종합 답변.
- 논문 제목 **From Local to Global**의 뜻: 낱낱의 관계(local) → 무리 요약(community) → 전체 종합(global).

6-7. 냉정한 비용

청크마다 LLM로 관계 추출 + 그래프 전체를 무리별로 LLM 요약. **GraphRAG가 가장 비싼 RAG**. 전역 질문(보고서 요약·주제 분석)이 실제 핵심 가치일 때만 정당화된다.

7. 한 장 치트시트

RAG = "검색해서 찾은 자료를 붙여 LLM이 답하게 하는 것"

[세션 1 색인 · 미리] 문서->파싱->청킹->임베딩->벡터DB(+ANN지도)

[세션 2 질의 · 답변] 질문->임베딩(같은모델)->ANN검색->[질문+청크 텍스트]->LLM->답

임베딩 = 의미를 숫자로 '만들기' (분포가설: 맥락으로 의미 파악)

코사인 = 두 벡터를 '재기' (방향만 봄, 길이는 무시)

ANN = 근사검색 (정확도 조금 내주고 속도 폭증) -> 벡터DB=검색엔진

철칙 = 색인 · 질의 임베딩 모델은 반드시 동일 / LLM엔 텍스트가 들어감

GraphRAG = 답을 대신 '연결'을 따라 걷는 RAG

저장소 2개(벡터+그래프) / 추출+개체통합 / local->global(커뮤니티 요약) / 비쌈

고도화 순서 = 0.평가 -> 1.재랭킹+혼합검색 -> 2.질문변형 -> 3.자기교정 -> 4.구조전환

8. 다음: 5주차 — 평가·하네스 (여기서 이어가면 됨)

핵심 질문: RAG를 고쳤다. 그래서 좋아졌나 나빠졌나 — 느낌 말고 '숫자로' 어떻게 증명하나?

왜 어려운가: 일반 코드는 정답이 하나라 채점이 쉽다($2+2=4$). 그런데 LLM의 답은 정답이 여러 개다.

질문: "소화기 설치 기준은?"

답 A: "연면적 33㎡ 이상이면 설치."

답 B: "바닥면적이 33제곱미터를 넘으면 비치."

-> 글자는 다른데 뜻은 같다. 기계가 어떻게 '둘 다 정답'으로 채점하나?

이 "열린 텍스트 자동 채점" 문제를 못 풀면, 5장의 모든 고도화는 그냥 감(感)이다.

5주차 입구 질문(스스로 답해볼 것):

글자는 다른데 뜻은 같은 두 답을 기계가 '같은 정답'으로 인식하려면 무슨 기술을 쓰면 될까?

(힌트: 3주차에서 배운 그것 — 의미를 숫자로 바꿔 **코사인**으로 **닮음**을 재면 된다. 이걸 평가에 응용한 게 '의미 유사도 채점'이고, 더 나아가 'LLM을 채점관으로 쓰는 LLM-as-judge'로 확장된다.)

1~4주차 완료. 이 리포트의 끝점이 곧 5주치의 시작점이다.